

Reinforcement Learning for Knowledge Graph Co-optimization: A Dual-Strategy Rule Generation Framework

Anonymous Authors

Abstract—Knowledge-graph quality control couples two decisions: which constraints to acquire and which graph defects to repair under a limited budget. We formulate this interaction as a finite-horizon control problem and train a Double-DQN policy over four graph operations and four rule-management operations in a fixed validator registry. A dual-strategy module proposes text-derived rules through deletion completion and augmentation expansion; separate execution studies test their repair coverage. An action-level audit reveals that a count-based penalty for introduced violations suppresses relation repair because restoration can create temporary duplicates. We compare count, initial-size rate and zero penalties using forty neural models, ten fitted one-step controls, and separate development and test scenarios. On thirty unseen corruptions of the same base graph, rate-penalty Double DQN improves reference-triple F1 from 94.67% to 99.49% and restores 94.00% of the original invalid relations. A simple acquire-then-deficit heuristic reaches 99.91% F1; the study establishes reward sensitivity without a learned-policy advantage over the strongest simple controls. Replaying 9,456 archived responses shows complementary rule candidates on the same documents, while augmentation alone yields more constraints at equal call budgets. An execution audit compiles 18,143 exact typed patterns; their union detects 10 of 64 defects in RuleTest-94. An offline bridge removes ten prohibited records while retaining all thirty clean records. Document-based development tests expose limited repair coverage before learned scheduling with generated rules. The results link reward design to repair behavior and identify executable rule coverage as a condition for extending the learned loop.

I. Introduction

A. Research Background and Motivation

Knowledge graph (KG) quality control involves two linked decisions: which defects to repair and which rules to acquire. A repair may require a validator that is not yet available. Acquiring more rules consumes resources, and a rule that covers few remaining defects may have little immediate value. This dependence motivates a scheduler that responds to the current graph and available rules.

Rules also require a source of new candidates. Expert constraints capture known requirements, while text can suggest additional type, hierarchy, and procedural constraints. Large language models (LLMs) offer a way to propose such candidates. Turning a proposal into a useful validator requires executable semantics, a compatible vocabulary, and evidence of detection quality.

We study reinforcement learning (RL) for allocating a finite budget between graph and rule operations, together with two ways to elicit rule candidates from text. The central link is an

executable validator: acquiring it changes which repairs are available, and repairing the graph changes the value of later acquisitions.

Our evaluation follows this link in three stages. First, a fixed validator registry lets us compare learned scheduling, simple policies, and reward choices under the same graph transitions. Second, a paired generation archive measures what deletion completion and augmentation expansion contribute. Third, direct execution and document-based development tests check whether generated candidates supply useful repair actions. The trained policy results come from the first stage; the third stage uses fixed schedules to assess the conditions for learning with generated-rule arrivals.

B. Contributions of This Paper

We make three contributions:

- 1) **An executable graph–rule co-optimization environment.** We define fourteen observed features, eight graph and rule actions, feasibility masks, and a cost-aware reward. Graph actions change stored records, and rule actions change the validators available for later repairs.
- 2) **A trained Double-DQN policy and matched control study.** An action-level audit shows how a count penalty can discourage useful relation repairs by charging for temporary duplicates. Forty retrained neural policies, ten fitted one-step baselines and reserved test scenarios compare three fixed penalties using reference-fact and structural metrics. Matched ablations under the rate reward test graph features, rule features, action masking and acquisition cost. The studies identify reward sensitivity, the contribution of masking, and the remaining performance gap to simple controls.
- 3) **Dual-strategy rule generation with traceable evaluation.** Deletion completion and augmentation expansion propose candidates in a shared schema. We compare candidate diversity at equal call budgets, distinguish declarations from constraints, and trace exact typed-rule execution back to the archived response. Fixed-schedule execution and source-evidence development tests then locate the gap between candidate diversity and useful repair coverage.

Figure 1 summarizes the framework. The policy receives fourteen features describing graph quality, rule quality, remaining defects, remaining budget, and active rule fractions. A feasibility mask controls action selection. The selected

operation changes the environment, which recomputes quality and reward. Separate diagnostic probes inspect one-step transitions; evaluated policies use only the current observation and mask.

C. Paper Organization

Section II reviews related work. Section III presents the scheduling model and rule-generation procedures. Section IV follows the three evaluation stages, then discusses their joint implications. The appendices provide protocols, detailed execution records, and supporting repair-layer checks.

II. Related Work

A. Reinforcement Learning for Knowledge Graphs

MINERVA by Das et al. [1] learns policies for multi-hop query answering. DeepPath by Xiong et al. [2] uses reinforcement learning to discover reasoning paths. Lin et al. [3] study reward shaping for multi-hop reasoning. These methods select graph traversal actions to answer queries. Our action space instead changes graph records and acquires validators that enable later repairs.

The scheduling question is whether a learned value function can allocate a limited budget better than a fixed schedule or an observation-based heuristic. We compare these alternatives on the same executable transitions and use retrained ablations to test the roles of observed features and action masking. The controlled registry lets us reproduce each transition, while the text-derived rule study examines the candidate source separately.

B. Language Models for Knowledge Engineering

KG-BERT [4] uses a language model for triple classification, and KEPLER [5] combines language modeling with knowledge embeddings. Pan et al. [6] review ways to connect language models and knowledge graphs. These studies motivate using text as a source of information for graph construction and completion.

Chain-of-thought prompting [7] and ReAct [8] explore structured reasoning and tool use. Our generation module uses two single-call elicitation tasks with a common output schema: deletion asks which constraints are exposed by missing spans, while augmentation asks which constraints are suggested by supplementary clauses. We evaluate the resulting candidates through archived provenance and executable patterns, rather than treating a reasoning response as a validated rule.

C. Constraint Languages, Rule Mining, and Repair

SHACL [9] and ShEx [10] express graph constraints. Their detection coverage depends on the shapes provided. A limited set of schema checks should therefore be distinguished from the expressiveness of the constraint language itself.

AMIE [11] mines Horn rules from graph facts, RuDiK [12] studies rule discovery in knowledge bases, and Neural LP [13] learns logical rules through differentiable operations. Our candidates originate from source documents and include type

patterns and procedural statements. Their different representation requires a validation and compilation step before they can act as graph validators.

Lin et al. [14] systematically evaluate LLM-based graph repair. This makes controlled constraint and repair comparisons directly relevant to our study. We focus on the order of validator acquisition and graph edits, with an additional provenance-based audit of generated candidates. An end-to-end comparison with external repair and rule-mining methods requires aligning executable constraints, input evidence, and evaluation budgets.

III. Methodology

A. Framework Overview

The framework maintains a graph G_t and a rule set R_t . At each step, a policy chooses whether to acquire or prune rules, or to repair the graph using an active validator. Rule acquisition can enable later repairs, while the remaining graph defects determine the value of further acquisition. Our research question concerns this allocation of a finite decision budget.

The implementation has a scheduling loop and a candidate-generation module, connected through executable validators. Figure 1 shows the loop; Figure 2 shows how the two generation strategies produce candidates for validation. We use a fixed registry to study scheduling under controlled transitions, then compile archived candidates and test their effects through an offline execution interface. Document-based development tests further examine response-level acquisition and source constraints. These settings share the acquisition–validation–repair dependency, while using different actions and rule inputs.

B. Quality Assessment Model

1) *Graph Quality*: After every graph operation, deterministic checks compute isolation, exact-duplicate relation, schema-invalid relation, and dangling-edge rates, denoted I_r, D_r, V_r, L_r . Their quality components are

$$\begin{aligned} S(C_1) &= 1 - I_r, & S(C_2) &= 1 - D_r, \\ S(C_3) &= 1 - V_r, & S(C_4) &= 1 - L_r. \end{aligned} \quad (1)$$

The controlled environment uses

$$Q(G) = 0.20S(C_1) + 0.20S(C_2) + 0.30S(C_3) + 0.30S(C_4). \quad (2)$$

These fixed weights define the benchmark objective. They do not estimate the factual correctness of arbitrary triples.

2) *Rule Quality*: Active validators are scored on a fixed 180-case calibration inventory, separate from the graph corruption manifest. Their union prediction determines

$$\text{Precision}(R) = \frac{TP}{TP + FP}, \quad \text{Recall}(R) = \frac{TP}{TP + FN}. \quad (3)$$

Both are zero for an empty active set in the environment. Coverage is the fraction of four registered defect families with an active validator. The online score is

$$\begin{aligned} Q_{\text{online}}(R) &= 0.35 \text{Precision}(R) + 0.35 \text{Recall}(R) \\ &\quad + 0.30 \text{Coverage}(R). \end{aligned} \quad (4)$$

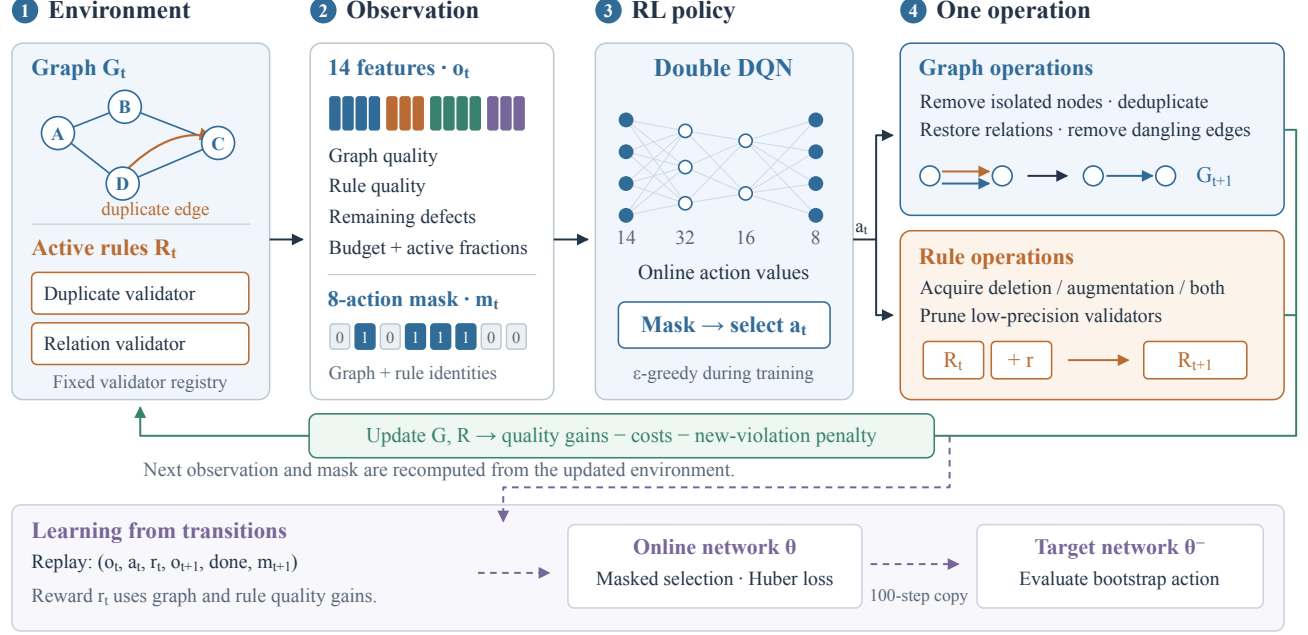


Fig. 1. RL graph-rule co-optimization. The environment supplies fourteen observed features and an eight-action feasibility mask. The policy selects one graph or rule operation; the environment then updates quality and reward. The lower band shows replay-based Double-DQN training. Miniature graphs and mask entries illustrate the mechanism. The scheduling loop uses a fixed validator registry; text-derived candidate generation is shown in Figure 2.

The calibration predictions are prescribed by the controlled benchmark: each high-precision module detects 27 of 30 family positives and two clean cases. Three additional modules provide lower-precision alternatives. These scores describe that inventory, rather than estimates from a live production stream. Candidate counts from the generation archive are measured separately from rule precision and recall.

C. RL-Based Graph-Rule Co-Optimization

1) *Environment and Policy Observation*: The underlying finite-horizon state contains the graph, active rules, remaining budget, initial defect counts, and the environment’s trusted relation-restoration records. These records are private to the environment. The policy receives a 14-dimensional observation:

$$o_t = [S(C_1), S(C_2), S(C_3), S(C_4), \text{Precision, Recall, Coverage, } \tilde{n}_{\text{iso}}, \tilde{n}_{\text{dup}}, \tilde{n}_{\text{invalid}}, \tilde{n}_{\text{dang}}, \tilde{b}, \rho_{\text{valid}}, \rho_{\text{noisy}}]_t. \quad (5)$$

Violation counts are divided by their episode-initial counts, with denominator at least one. The final terms give the remaining budget fraction and active fractions of the predefined high- and low-precision modules. The observation is clipped to $[0, 1.5]$ componentwise. Coverage and ρ_{valid} coincide in this registry; both entries are retained for matched architectural comparisons. This compressed observation need not be a sufficient Markov state. The network uses it as a value-function input, together with an externally computed feasible-action mask.

The eight actions are

$$\mathcal{A} = \{a_{\text{iso}}, a_{\text{dup}}, a_{\text{rel}}, a_{\text{dang}}, a_{\text{del}}, a_{\text{aug}}, a_{\text{mix}}, a_{\text{prune}}\}. \quad (6)$$

The first four remove isolated noise nodes, deduplicate relations, restore invalid relation names, and remove dangling edges. Each processes up to a batch determined by 60% of the remaining matching defects. Relation restoration uses the private corruption record after detection. The other actions activate a deletion-family module, activate an augmentation-family module, activate one available module from each family, or prune modules with calibration precision below 0.5. Module acquisition follows the fixed registry order. A repair is feasible only if its validator is active and matching defects remain. The deletion and augmentation action names denote predefined module families in this benchmark; taking these actions activates stored validators. The text-generation procedures below produce candidate rules for the separate execution studies.

The mask is computed from the current graph and active rule identities. It is not necessarily recoverable from the fourteen observation values alone. All matched policy baselines receive the same mask, except the explicit no-mask ablation. Policies cannot access the private corruption record or query future transitions; the separate model-informed reference can clone and probe the environment.

2) *Reward and Double DQN*: The reward uses $\Delta Q(G_t) = Q(G_{t+1}) - Q(G_t)$ and the analogous rule-score change, with costs for acquisition, edits, and new violations:

$$r_t = \Delta Q(G_t) + 0.4 \Delta Q_{\text{online}}(R_t) - 0.004 C_{\text{call}} - 2 \times 10^{-5} C_{\text{edit}} - P_t. \quad (7)$$

C_{call} counts one unit for either single-family acquisition and two for mixed acquisition. It is an accounted model-call cost, not an actual API request. Pruning has no call cost. The revised environment computes N_{new} from violation identities before and after the action, rather than returning a fixed zero. Nodes have persistent identifiers; each edge occurrence has its own identifier, including duplicate occurrences. The edit counter includes graph mutations and changes in active rules; we also report these two components separately. An episode ends when $Q(G) \geq 0.999$ and $Q_{\text{online}}(R) \geq 0.90$, after 18 decisions, or when no feasible action remains. A destructive proposal that would clear a nonempty node or edge set is rolled back and ends the episode.

Let $\mathcal{V}(G)$ be the multiset of detected violation identities. Then

$$N_{\text{new}} = |\mathcal{V}(G_{t+1}) \setminus_m \mathcal{V}(G_t)|, \quad (8)$$

where $\setminus_m$ is multiset difference. Removing a dangling edge can isolate its remaining endpoint: one old violation disappears and a different one appears, although the total count stays unchanged. The new term charges for that introduced violation. Each transition stores the introduced identities and all reward components. We compare three fixed definitions of P_t . Let $n_{k,t}$ count introduced identities of violation family k , and let w_k be its weight in Equation 2. The penalties are

$$\begin{aligned} P_t^{\text{count}} &= 0.01 \sum_k n_{k,t}, \\ P_t^{\text{rate}} &= \sum_k w_k \frac{n_{k,t}}{D_{k,0}}, \quad P_t^{\text{zero}} = 0, \\ D_{k,0} &= \begin{cases} \max(1, |V_0|), & k = \text{isolation}, \\ \max(1, |E_0|), & \text{otherwise.} \end{cases} \end{aligned} \quad (9)$$

Here V_0 and E_0 are the nodes and edge occurrences after initial corruption. Denominators remain fixed throughout an episode, so deleting records cannot change the scale of later penalties. The rate variant charges for an introduced violation relative to the initial graph size. It leaves the call and edit costs unchanged and therefore does not make the full reward scale-invariant. All three variants use the same observations, masks and transition operators. A duplicate occurrence is a structural violation but need not be a new wrong fact; we measure both outcomes separately.

For $J_t = Q(G_t) + 0.4Q_{\text{online}}(R_t)$, discounted quality gains satisfy

$$\begin{aligned} \sum_{t=0}^{T-1} \gamma^t (J_{t+1} - J_t) &= -J_0 + \gamma^{T-1} J_T \\ &\quad + (1 - \gamma) \sum_{t=1}^{T-1} \gamma^{t-1} J_t. \end{aligned} \quad (10)$$

Thus the policy values earlier improvements as well as terminal quality.

We use DQN [15] with the Double-DQN target [16]. The online Q-network has architecture $14 \rightarrow 32 \rightarrow 16 \rightarrow 8$ with ReLU activations. A replay buffer stores $(o_t, a_t, r_t, o_{t+1}, d_t)$

Algorithm 1 Archived deletion-completion procedure

Require: Document identifier d , source text T

- 1: $(T^-, F) \leftarrow \text{DeleteShortSpans}(T)$
 - 2: $P \leftarrow \text{DeletionPrompt}(T, T^-, F)$
 - 3: $Y \leftarrow \text{LLM}(P)$ $\triangleright$ One call
 - 4: Parse the JSON candidate categories in Y
 - 5: $\text{Log}(d, \text{deletion}, F, Y)$
 - 6: **return** categorized candidates, including parse status
-

and the next-action mask. For a nonempty next-action set, a^* selects the online-network maximizer. The target is

$$\begin{aligned} a^* &= \arg \max_{a' \in \mathcal{A}_{t+1}} Q(o_{t+1}, a'; \theta), \\ y_t &= \begin{cases} r_t, & d_t = 1 \text{ or } \mathcal{A}_{t+1} = \emptyset, \\ r_t + \gamma Q(o_{t+1}, a^*; \theta^-), & \text{otherwise,} \end{cases} \\ \mathcal{L}(\theta) &= \mathbb{E}[\text{Huber}(Q(o_t, a_t; \theta) - y_t)]. \end{aligned} \quad (11)$$

The bootstrap term is zero for terminal states or an empty feasible set. Target parameters are copied every 100 environment steps. Standard DQN changes only the next-state target to $\max_{a'} Q(o_{t+1}, a'; \theta^-)$.

We use Adam with learning rate 0.002, discount 0.95, replay capacity 20,000, batch size 64, and 128 transitions of warm-up. Each model trains for 250 episodes. The archived configuration decays exploration linearly over 260 episodes, so the last training episode uses $\epsilon \approx 0.0902$, rather than reaching the lower limit of 0.05. The reward study retains ten checkpoints for each of four settings: Double DQN with count, rate or zero penalty, and standard DQN with rate penalty. The earlier sixty-model component study uses the count penalty and is reported separately in Appendix E. Training uses the local environment and makes no API calls.

D. LLM-Based Dual-Strategy Rule Generation

The two strategies propose source-based candidates that can extend a rule inventory. The stored implementation uses one generation call per document and strategy. Its JSON response contains entity-type declarations, relation-type declarations, allowed and forbidden type-relation patterns, hierarchy statements, geographical hierarchy patterns, and procedural statements. Declarations and constraint candidates are counted separately in the matched-budget analysis.

1) *Deletion Completion*: The implementation removes short character spans, rather than entity-level masks. It selects up to five start points, with each span at most six characters under the script defaults, and replaces them with a missing-span marker. The prompt supplies the original text, modified text, removed fragments, and a shared JSON schema. It asks the model to describe missing rule information and propose reusable constraints. Providing the original text makes this an explicit contrastive elicitation task; it is not a blind reconstruction test.

2) *Augmentation Expansion*: The augmentation prompt supplies the source and asks for up to three plausible supplementary clauses under the script default. The same response proposes constraints suggested by those clauses. It does not

Algorithm 2 Archived augmentation-expansion procedure

Require: Document identifier d , source text T

- 1: $P \leftarrow \text{AugmentationPrompt}(T)$
- 2: $Y \leftarrow \text{LLM}(P)$ $\triangleright$ Clauses and candidates in one call
- 3: Parse the JSON candidate categories in Y
- 4: Log $(d, \text{augmentation}, Y)$
- 5: **return** categorized candidates, including parse status

run a separate graph extractor on each variation. The generated clauses provide exploration cues; they are not assumed to be independent samples from the real document distribution.

3) *Rule Materialization and Validation*: A candidate is not automatically an executable or correct rule. We preserve the source document, strategy, response line, candidate category, and ordinal position. The direct execution audit supports explicit triples of subject type, relation, and object type. A forbidden pattern $r = (\tau_s, p, \tau_o)$ flags a record x when

$$V_r(x) = \mathbf{1}[(\text{type}(s_x), p_x, \text{type}(o_x)) = r]. \quad (12)$$

An allowed pattern records permission for that exact combination. Absence from the allowed set does not imply a violation. When both allowed and forbidden patterns match, the audit records a conflict and abstains. Only outer whitespace is trimmed; no aliases or semantic rewrites are inferred.

Each malformed candidate, unsupported family, unmapped vocabulary item, and unmatched executable pattern remains in the audit. The executable subset is checked on RuleTest-94 using its stored expected outcomes. These labels describe the designed suite and provide an execution check, not independent adjudication of generated rule meaning. The procedural and structural family detectors are a separate hand-implemented comparison and are not attributed to individual generated candidates.

4) *Aggregation and Provenance*: Candidates are deduplicated within category by the stored aggregation code and canonical serialization. Entity and relation declarations are distinguished from constraints. The replay uses category-level canonical deduplication; it makes no additional semantic merges.

The generation script defaults to Qwen3-32B and temperature 0.2. Per-call records do not preserve a complete run configuration, exact rendered prompt, or decoding metadata. We therefore report those values as script defaults, not verified settings for every historical call. The prompt builders and translated templates are documented in Appendix A; literal templates are included in the supplementary artifact. The independent extraction benchmark uses Qwen3.8-27B-no-thinking and is separate from rule generation.

E. Offline Generated-Rule Bridge

The bridge compiles archived allowed and forbidden typed patterns using the same exact-match semantics as the execution audit. Activation changes the current violation report and enables a removal action when an unopposed forbidden pattern matches a record. After removal, the bridge recomputes matches and action availability. Every match retains

its rule identifier and source lineage. Unsupported candidates are logged; conflicting permissions and prohibitions cause abstention.

The separate replay adapter exposes reset, observation, available-action and step interfaces. Its four commands load the deletion bank, load the augmentation bank, remove forbidden matches, or stop. A policy receives aggregate record, flag, conflict and active-rule counts, acquisition status, remaining budget and the current mask. Each bank is loaded at most once; stop or a six-decision limit ends replay. This adapter has no reward output and does not replace the eight-action training environment.

Records with missing endpoints or unknown types cause abstention. A permission that arrives after deletion is logged against the removed record, without silently recreating it. A removal that would empty a nonempty record set is blocked. The adapter stores each activation, detection and removal with source lineage. Reference labels enter only the scorer; no replacement facts are inferred. Loading a complete archive bank is measured separately from model calls.

a) *Response-level development environment*: The document-based extension acquires one cached response at a time, using four actions: acquire deletion, acquire augmentation, repair, and stop. It allows four response acquisitions and ten decisions per episode. Its reward separates violations revealed by new rules from violations caused by editing (Appendix C-A). This extension introduces response-level acquisition and its own reward for the document task. Fixed schedules and exhaustive replay assess its repair coverage before policy training; the eight-action checkpoints are evaluated only in their original registry environment.

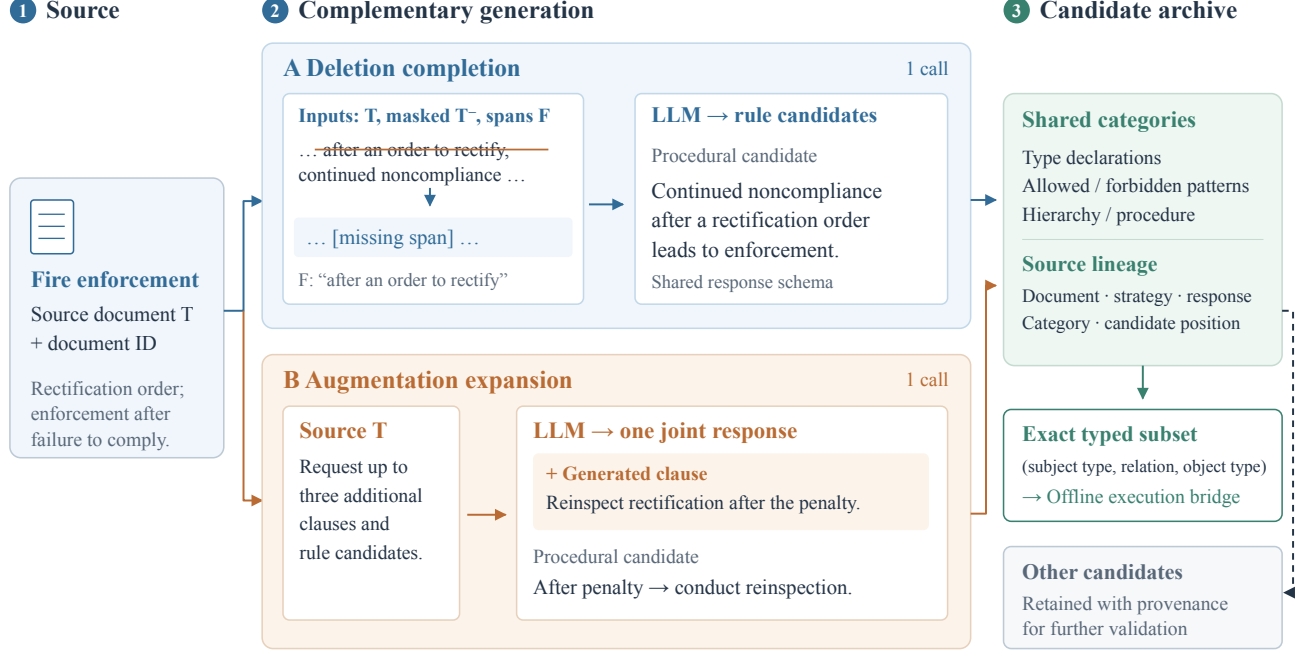
IV. Experiments

A. Evaluation Design

1) *Research Questions*: We evaluate the framework in the order in which a rule becomes useful: a scheduler acquires a validator, a generation strategy supplies a candidate, and execution determines whether that candidate enables a repair. **RQ1**: How does learned scheduling compare with simple policies when the available validators and graph transitions are fixed? **RQ2**: Which reward terms and policy components affect repair completion and fact preservation? **RQ3**: What do the two generation strategies contribute at equal call budgets, and do their candidates enable complementary repairs?

The fixed-registry experiments answer RQ1 and RQ2. The generation archive, execution bridge, and document-based development tests answer successive parts of RQ3. The last stage evaluates fixed schedules and candidate coverage; learned scheduling with generated-rule arrivals remains the integration step to be established.

2) *Data and Protocol*: The policy environment uses 450 balanced TNEWS/CLUE documents, 30 per category, to build a graph with 873 nodes and 1,083 relations. Independent seeds control isolation, duplication, invalid-relation, and dangling-edge corruption. The reward study reserves twenty development scenarios and thirty test scenarios; all share the same base



Example: shortened English translations from one archived fire-enforcement document; generated clauses are proposals.

Fig. 2. Dual-strategy rule generation with shortened English examples from one archived fire-enforcement document. Deletion completion receives the original text, masked text and removed spans. Augmentation expansion returns supplementary clauses and rule candidates in one response. Both strategies preserve candidate provenance. Exact typed patterns enter the offline execution bridge; other candidates remain available for further validation.

documents and graph. The earlier ten scenarios are used for the action audit and historical component study. The separate 180-case calibration inventory supplies online validator precision and recall. Rule acquisition activates registered modules; relation restoration uses the environment’s private source record after a validator detects an invalid relation.

The rule study replays 9,456 successful archived responses, with deletion and augmentation responses for each of 4,728 government-service documents. RuleTest-94 supplies 30 clean and 64 defective records with stored expected outcomes. It is a designed test suite. DocRED provides given entity types and source text for the response-level development tests (Appendix C). Separate structural and extraction checks appear in Appendix G.

3) *Reproducibility*: Training sources, seeds, costs, and episode budgets are frozen before fitting the reward-study models. A second manifest seals the models and scorer before the reserved test evaluation. All predefined policies and failures are retained. Policies receive public observations and masks; reference labels are used for scoring. Appendix D gives the split and inference protocol, and Appendix F lists the archived transitions and reproduction entry points. Local policy training and cached replay make no API requests; generation costs are reported separately.

B. RL Co-optimization on Executable Graph Transitions

1) *Reward Audit*: The earlier count-penalty study reported zero introduced violations for Double DQN. Replaying its ten trajectories reveals zero relation-repair actions and 61.3 invalid relations remaining per graph. We therefore branch

from each visited state to inspect feasible actions. Across twenty earlier Double-DQN and heuristic trajectories, the audit executes 2,214 one-step branches, including 311 relation-repair branches. These simulator probes are diagnostic and are never available to the evaluated policies.

All 155 feasible relation-repair branches along the earlier Double-DQN trajectories have negative immediate reward under the count penalty. They introduce 443 duplicate occurrences in total across these overlapping branches, but add no wrong reference facts and remove no correct facts. Their mean immediate return changes from -0.01990 under the count penalty to $+0.00821$ under the rate penalty. Repair followed by deduplication gives mean two-step discounted returns of -0.01781 and $+0.01030$, respectively. The original penalty thus discourages a useful repair because of its temporary duplicates. Branch totals describe the audited states, not independent test samples.

2) *Matched Policies and Measures*: The new comparison trains Double DQN under each of the three penalties and DQN under the rate penalty. All eight evaluated policies use the same fourteen observed features, feasible-action mask and eighteen-step horizon. The *acquire-then-deficit* heuristic first acquires missing high-precision modules in mixed, deletion, then augmentation priority. It then chooses the feasible graph action with the largest weighted current quality deficit. Random-valid and rule-first-valid schedules provide additional controls.

A one-step ridge baseline fits three outcomes per action: joint quality gain, edit count, and the introduced-violation rate burden. Each of its ten fits uses 250 independent episodes of

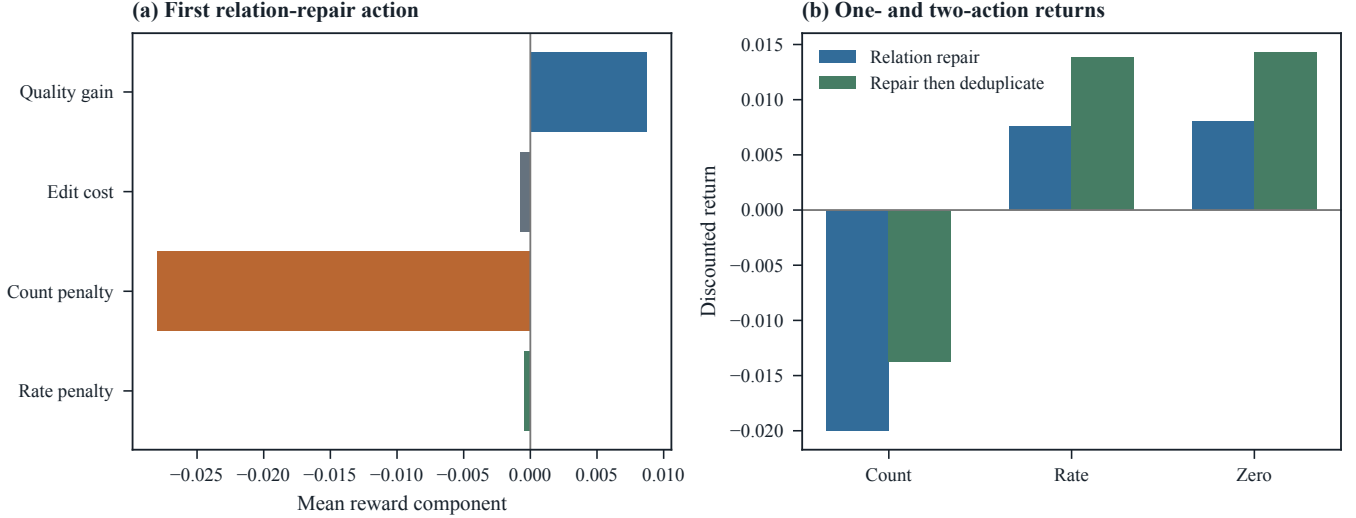


Fig. 3. Reward decomposition on ten earlier scenarios. Each probe starts from the initial corrupted graph with all four valid detectors active. (a) Mean quality gain, edit cost and alternative penalties for the first relation repair. (b) Returns for that action and for repair followed by deduplication, with discount 0.95. The two penalties in (a) are alternatives, not additive terms.

uniform feasible-action exploration, the same episode budget as each neural model. Inputs are the fourteen public features; targets are observed reward components. Standardization is fitted separately per action on training data, with ridge coefficient one and an unpenalized intercept. At inference, predicted edits and burden are clipped at zero. The policy chooses the feasible action with highest predicted rate-penalty reward, including the known call cost. Its data come from separate random trajectories and its supervision uses reward components, so equal episode budgets do not mean identical training data or feedback. No policy receives reference labels or probes future transitions.

Let F^* , F_0 and F_T be unique triple sets from the clean reference, initial corrupted graph and final graph. Our primary reference metric is

$$F1_{\text{triple}} = \frac{2|F_T \cap F^*|}{|F_T| + |F^*|}. \quad (13)$$

The second primary metric is the fraction of initially invalid edge identities still present with their reference relation restored. Deleting an invalid edge earns no restoration credit; a repaired occurrence later removed by deduplication also earns none. Triple-set F1 measures retained facts independently of this stricter occurrence-level measure. Correct-fact preservation is $|F_T \cap F_0 \cap F^*| / |F_0 \cap F^*|$. These references come from controlled corruption of deterministic extraction, rather than independent semantic annotation.

We also report residual defects, introduced violations by family, correct and wrong fact changes, edits, acquisition units, final joint quality and common-budget trajectory quality:

$$\bar{Q}_t = \frac{Q(G_t) + 0.4Q_{\text{online}}(R_t)}{1.4}, \quad (14)$$

$$AUC_{18} = \frac{1}{18} \sum_{t=0}^{17} \frac{\bar{Q}_t + \bar{Q}_{t+1}}{2}.$$

Early terminal quality is carried forward to step eighteen. All evaluation returns use the rate penalty, even for policies trained

under count or zero penalty; the archive also stores both counterfactual returns. Acquisition units measure simulated cost, and actual API requests are zero.

3) *Policy Results*: Table I shows that count-penalty Double DQN again takes no relation-repair actions, leaving 60.03 invalid relations per graph. Its triple F1 is 94.67%. Rate-penalty Double DQN restores 94.00% of the original invalid edge occurrences and reaches 99.49% F1, a gain of 4.824 percentage points with a 95% interval of [4.403, 5.141]. Both primary count-versus-rate contrasts pass Holm correction ($p = 0.0156$). The rate policy averages 3.92 relation-repair actions and leaves 3.57 invalid relations per graph.

The zero-penalty policy reaches 99.75% F1 and 99.06% restoration. Ridge reaches 99.88% and 97.90%; acquire-then-deficit reaches 99.91% and 98.77%. Rate-penalty Double DQN has lower means on both metrics than these three controls. None of those six paired contrasts passes the eight-test Holm correction (Table II). The comparisons establish an improvement over the original count penalty, without establishing an advantage over zero penalty or the simple controls. The rate policy’s restoration standard deviation across seed means is 12.44 percentage points; Figure 4 shows this training variability.

All eight policies retain every initially correct reference fact and introduce no new wrong reference fact. Newly introduced violations are duplicate occurrences. Rate-penalty Double DQN introduces 4.20 per graph, compared with 4.42 for zero penalty and 4.37 for the heuristic. Under the common rate reward, mean discounted returns are 0.3806, 0.3803 and 0.3877, respectively. The heuristic also uses fewer acquisition units: 4.00 versus 4.42 for rate-penalty Double DQN. In this registry, both the simple fitted policy and the fixed heuristic remain strong controls.

4) *Inference and Earlier Component Ablations*: The test analysis first averages the thirty scenarios within each training or run seed. Ten paired seed means, rather than 300

TABLE I

Reward validation on thirty unseen corruption scenarios of the same base graph. Means and standard deviations are over ten run-seed means. F1 uses unique reference triples; restoration requires the original erroneous edge to regain its reference relation. All returns use the rate penalty.

Policy	Triple F1 (%)	Restored (%)	Preserved (%)	New viol.	Calls	Return
DDQN (count)	94.67 $\pm$ 0.00	0.00 $\pm$ 0.00	100.00	0.00	6.27	0.3692
DDQN (rate)	99.49 $\pm$ 0.64	94.00 $\pm$ 12.44	100.00	4.20	4.42	0.3806
DDQN (zero)	99.75 $\pm$ 0.31	99.06 $\pm$ 1.37	100.00	4.42	4.06	0.3803
DQN (rate)	99.77 $\pm$ 0.34	96.94 $\pm$ 6.28	100.00	4.33	4.21	0.3818
One-step ridge	99.88 $\pm$ 0.01	97.90 $\pm$ 0.15	100.00	4.37	4.04	0.3874
Acquire + deficit	99.91 $\pm$ 0.00	98.77 $\pm$ 0.00	100.00	4.37	4.00	0.3877
Rule-first valid	99.40 $\pm$ 0.00	94.73 $\pm$ 0.00	100.00	4.27	8.00	0.3514
Random valid	97.84 $\pm$ 0.19	70.72 $\pm$ 3.03	100.00	3.12	9.72	0.3394

TABLE II

DDQN with rate penalty minus comparator, in percentage points. Intervals resample ten run-seed means; each mean covers the same thirty scenarios. Exact sign-randomization tests are Holm-adjusted over eight contrasts.

Comparator	Metric	Difference	95% interval	Holm <i>p</i>
DDQN (count)	Triple F1	+4.824	[+4.403, +5.141]	0.0156
DDQN (count)	Restoration	+94.000	[+85.976, +99.919]	0.0156
DDQN (zero)	Triple F1	-0.255	[-0.673, +0.094]	0.9375
DDQN (zero)	Restoration	-5.057	[-12.825, +0.509]	0.9375
One-step ridge	Triple F1	-0.387	[-0.810, -0.069]	0.1875
One-step ridge	Restoration	-3.898	[-11.927, +2.006]	0.9375
Acquire + deficit	Triple F1	-0.415	[-0.837, -0.099]	0.1875
Acquire + deficit	Restoration	-4.772	[-12.796, +1.147]	0.9375

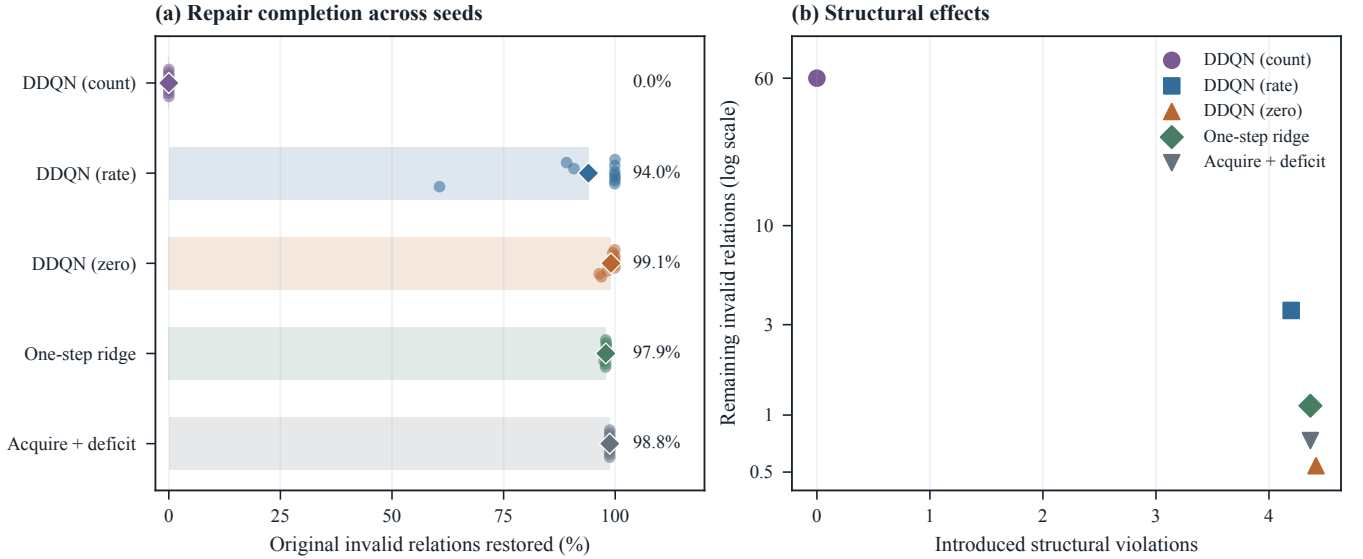


Fig. 4. Behavior on thirty unseen corruption scenarios. (a) Original invalid edge occurrences restored and retained; small dots show all ten seed means and diamonds show their mean. (b) Introduced structural violations and remaining invalid relations. Each seed mean averages thirty scenarios. Panel (b) shows the mean across seeds and uses a logarithmic vertical axis.

independent scenes, are the units for 10,000 paired-bootstrap draws and exact two-sided sign randomization over all 2^{10} assignments. Holm correction covers eight prespecified contrasts: rate-penalty Double DQN against count, zero, ridge and acquire-then-deficit, for both primary metrics. Pointwise intervals describe seed variability conditional on this fixed test set. Deterministic schedules repeat the same outcome across run seeds.

Appendix E retains the earlier count-penalty component study. The following matched comparisons use the current rate reward to test the contribution of features, masking, acquisition

cost, and penalty magnitude.

5) *Components under the Rate Reward*: We add five settings under the current transitions: remove graph features, remove rule features, remove the action mask, remove the training call penalty, or use a calibrated small count penalty. Ten seeds and 250 episodes per setting give fifty new models. The ten existing rate-DDQN checkpoints supply the matched complete-policy reference. All settings are evaluated on thirty further corruption seeds of the same base graph, giving 2,100 policy-seed-scenario outcomes including the heuristic. No model is selected by these outcomes.

TABLE III

Current-reward component study. Values average ten seed means, each over thirty common scenarios. Calls are simulated acquisition units; actual API requests are zero.

Setting	F1	Restoration	Calls	Unavailable
Rate DDQN	0.9952	0.9377	4.45	0.00
No graph features	0.9941	0.8938	4.10	0.00
No rule features	0.9775	0.6682	7.39	0.00
No action mask	0.9702	0.5776	6.23	8.26
No call penalty	0.9854	0.7686	5.54	0.00
Small count penalty	0.9905	0.9214	4.89	0.00
Acquire-then-deficit	0.9991	0.9884	4.00	0.00

The feature removals zero the specified observation entries during training and evaluation. Other observations and the action mask remain available. The no-mask setting selects from all eight actions in both behavior and target updates. Unavailable graph actions can be no-ops, and repeated rule acquisition can incur call cost; the underlying operators are unchanged. The no-call-cost setting removes only that training reward term. Evaluation uses the common rate reward.

Removing the mask lowers F1 by 2.50 points (95% seed-bootstrap CI: $[-3.45, -1.62]$; Holm $p = 0.0234$) and produces 8.26 unavailable selections per episode. It is the only contrast passing correction across twelve F1/restoration comparisons. Removing rule features also lowers mean F1 and raises acquisition cost, but its corrected test does not establish a stable effect across seeds. Every setting preserves initially correct reference facts in this controlled environment. The heuristic again has the highest mean F1 (99.91%).

a) Penalty magnitude control.: We calibrate a fixed count coefficient using twenty development rollouts with uniformly sampled feasible actions. The coefficient is the total rate burden divided by the total number of introduced violations: $\alpha = 0.000164493$. This uses observed transitions, not reference facts or test outcomes. Small-count DDQN reaches 99.05% F1, compared with 99.52% for rate DDQN; the paired difference is -0.47 points (CI: $[-1.73, 0.46]$, Holm $p = 1$). At the base size, these results do not separate the benefit of rate normalization from a much smaller count coefficient.

6) Fixed Policies across Replicated Graph Sizes.: We form one, two, four and eight disjoint copies of the clean base graph, with separate node IDs. The largest clean input has 6,984 nodes and 8,664 edges. At each size, ten fixed policy seeds and five common corruption seeds compare rate, original-count and small-count DDQN with acquire-then-deficit, giving 800 outcomes. All models were trained only at the base size. This sweep repeats the same content; it tests fixed-policy transfer rather than new documents or a large connected KG.

On the heuristic trajectories, increasing from one to eight copies multiplies the fixed small-count burden by 7.06, compared with 0.88 for the rate burden. This measures how the penalties scale on matched behavior. Fixed-policy outcomes and same-trajectory penalties do not establish an advantage for rate-based learning across sizes; that would require training comparisons at those sizes.

TABLE IV

Separate sequential heuristic resource measurement, three repeats per size. Time includes environment construction and rollout with tracemalloc. Memory is peak traced Python allocation, excluding the prebuilt clean graph and native tensor storage; it is not total RSS.

Clean edges	Copies	Median seconds	Median peak MiB
1,083	1	0.881	1.32
2,166	2	1.839	2.89
4,332	4	4.126	5.46
8,664	8	8.805	11.35

TABLE V

Equal-call candidate comparison (means over 30 document permutations). Declarations are entity and relation types; constraints are candidate patterns or statements.

Calls	Strategy	Docs	Decl.	Constraints
100	Deletion	100	468	702
100	Augmentation	100	501	857
100	Dual	50	512	795
1000	Deletion	1000	2634	6044
1000	Augmentation	1000	2887	7787
1000	Dual	500	2970	7126
4000	Deletion	4000	7249	21299
4000	Augmentation	4000	7627	28407
4000	Dual	2000	8081	25901

C. Dual-Strategy Candidate Generation

1) Same-Document and Equal-Call Comparisons.: On the full paired archive, deletion yields 32,955 distinct categorized candidates, augmentation yields 41,683, and their union contains 71,656. These totals include entity and relation declarations. The union is 71.9% larger than augmentation alone but uses twice as many calls: 9,456 versus 4,728. It measures additional candidate coverage on the same documents.

To compare candidate diversity at equal cost, we fix call budgets of 100, 500, 1,000, 2,000, and 4,000. For each of thirty seeded permutations of the paired document inventory, a single strategy processes B documents once each; the dual strategy processes the first $B/2$ documents with both strategies. Thus call count is matched while document coverage differs. Canonical aggregation follows the archived code, with declarations separated from constraint candidates. Table V gives representative budgets; all budgets and samples are released.

Augmentation produces more distinct constraint candidates than the dual strategy at every matched budget. At 1,000 calls, the means are 7,787 versus 7,126. Dual generation provides complementary proposals for a given document set, but it does not show a candidate-count advantage per call in this archive. The next stage measures how these candidates translate into executable repairs.

D. From Generated Candidates to Repair Actions

Candidate diversity answers only the first part of RQ3. We next check whether the candidates can execute, whether the two strategies enable different repairs, and whether acquisition order offers a useful scheduling choice.

1) Direct Execution of Archived Patterns.: The archive contains 181,851 candidate occurrences. Exact allowed/forbidden

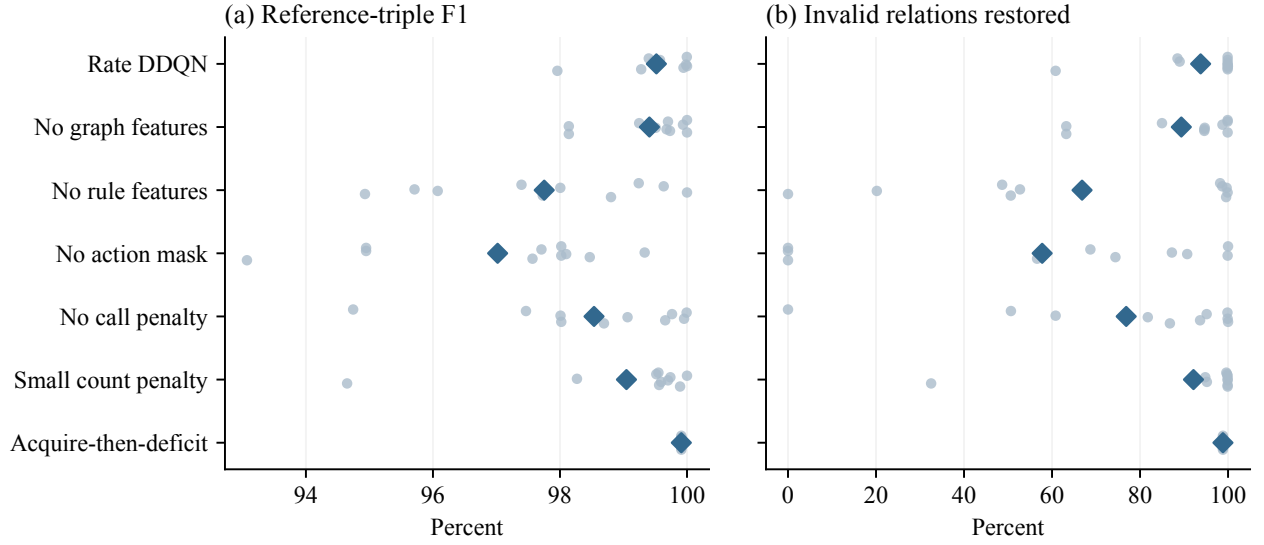


Fig. 5. Current-reward ablations and the calibrated count control. Small dots show all ten training-seed means; diamonds show their average. Each seed mean uses thirty shared corruption scenarios. The heuristic is deterministic.

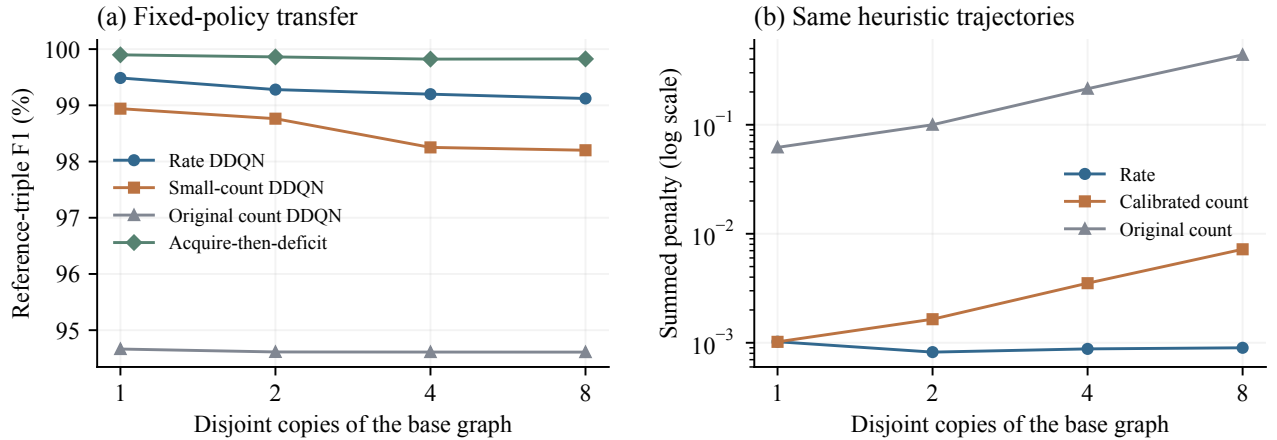


Fig. 6. Disjoint graph-size control. (a) Mean F1 over ten model seeds and five corruption scenarios at each size. (b) Three penalties evaluated on the same heuristic trajectories. Curves are descriptive; no model is retrained or selected by size.

TABLE VI

Direct execution of archived typed candidates on RuleTest-94. There are 64 defective and 30 clean cases.

Strategy	Patterns	TP	FP	Recall	F1
Deletion	5794	10	0	0.156	0.270
Augmentation	12617	0	0	0.000	0.000
Dual	18143	10	0	0.156	0.270

type patterns compile to 18,143 distinct rules, retaining the document, strategy, response line, category, and ordinal of each occurrence. Only two distinct compiled patterns match any RuleTest-94 record. Of 32,360 compilable occurrences, 32,348 contain vocabulary absent from the relevant suite positions, nine use represented vocabulary but match no record, and three match a record. This exposes a schema-alignment gap between the source archive and the test suite.

Direct execution of the union detects 10 of 64 defects and flags none of the 30 clean cases, giving recall 0.156 and F1 0.270. Augmentation alone makes no positive prediction, so its precision is undefined. Allowed patterns act as permissions rather than a closed-world whitelist; conflicting permissions and prohibitions cause abstention. No conflicts occur on this suite. Unsupported and malformed candidates remain in the audit counts and provenance files. Each strategy is scored on all 94 test cases.

2) *Generated Rules in an Offline Repair Loop*: The bridge in Section III-E activates the exact archived typed patterns on RuleTest-94, recomputes detections and the removal-action mask, executes allowed removals, and records the next state. Deletion activates 5,794 patterns, augmentation 12,617, and their union 18,143. Deletion and the union each remove ten defective records and no clean record; augmentation changes no record. All thirty clean records remain. The deleted records

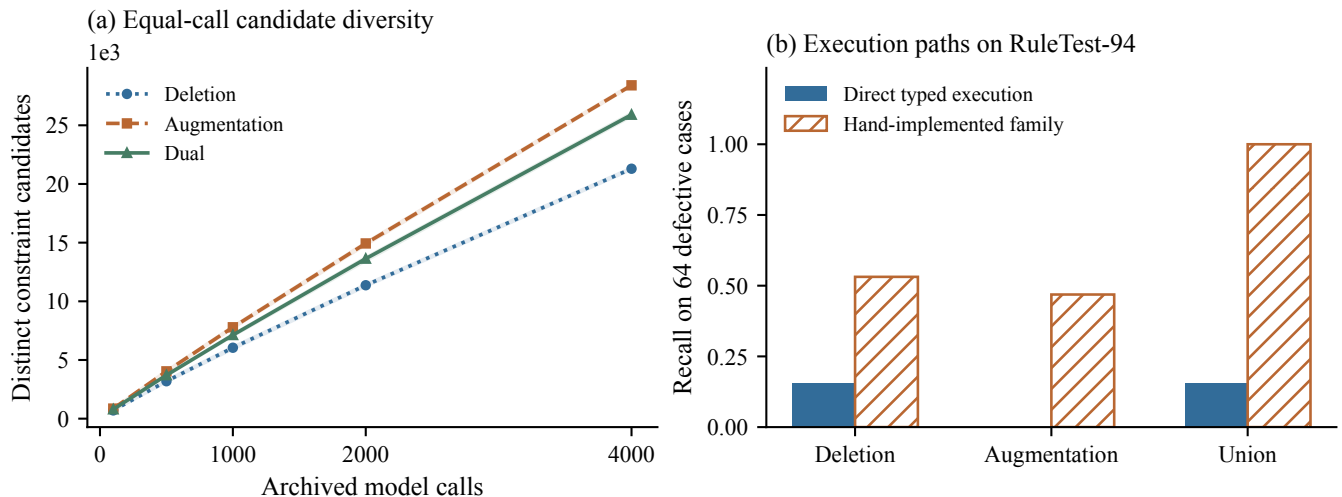


Fig. 7. Rule-generation analysis. (a) Distinct constraint candidates at equal call budgets; shading gives the empirical 2.5–97.5 percentiles across thirty document permutations, not an independent-sample confidence interval. (b) Recall from direct candidate execution and from the separate hand-implemented family detectors.

TABLE VII

Offline generated-rule bridge on the previously inspected RuleTest-94 suite. Removal resolves a typed prohibition; it does not reconstruct a reference fact.

Packet	Rules	Defect removals	Clean removals	Retained
Deletion	5794	10	0	84
Augmentation	12617	0	0	94
Union	18143	10	0	84

have no inferred replacements, so this is constraint removal rather than factual reconstruction.

Four fixed schedules vary acquisition order and whether removal is immediate or deferred. All reach the same RuleTest-94 result. The two matching patterns both come from deletion: one permission matches ten records and one prohibition flags another ten. Augmentation matches no record. Thus candidate diversity in the archive does not translate into complementary repairs on this suite.

The three raw graph snapshots supply a different check: all 26,270 edge occurrences lack the endpoint types required by the patterns, so the runtime abstains (Table VIII). These snapshots reveal an input requirement, while the typed suite reveals limited rule coverage. Appendix H gives the type-provenance audit and sixteen fixed-schedule traces. The bridge loads archived banks without a reward or learned policy; it establishes the rule-to-action interface.

3) *Response-Level Development Tests*: To test acquisition with document-specific responses, we use the four-action extension described in Section III-E. The initial typed DoCRED pilot compiles rules from both strategies but triggers no deletion. A source-evidence version then evaluates ten development pairs containing twenty documents, with given entity types and controlled injected records. It uses an original-source quote check and separates rule discovery from edit-induced violations in the reward. The expansion criterion requires source constraints to trigger repairs in at least two episodes and a fixed schedule to improve mean F1 while preserving at least 99% of reference facts. Appendix C-A gives

the protocol and both rounds.

The first round’s acquire-then-repair schedule raises F1 from 94.42% to 94.59% but preserves only 98.34% of reference facts, losing four. The second round, on the same records, reaches 95.69% without reference losses and removes seven of 31 injected items. All seven removals use deletion-generated type rules; no source-contradiction rule compiles. Neither round meets the full expansion criterion, so policy training and held-out evaluation in this extension remain unrun.

Exhaustive replay further distinguishes candidate quality from scheduling opportunity (Table XIII). In the first round, maximizing proxy return gives 91.63% reference F1, below the stop baseline. In the second round, the best feasible reference-scored schedule reaches 95.69%, equal to acquire-then-repair. These fixed candidates leave no additional final-F1 gain through scheduling. Acquisition order still changes discounted return and intermediate proxy quality; any cost or timing benefit requires its own evaluation.

E. Runtime and Cost Accounting

The scheduling studies use local environment transitions. Their acquisition units are simulated costs, reported alongside zero actual API requests. The archive analysis compares diversity per successful response: historical tokens, retries, failures, and latency are incompletely recorded. The two source-evidence development rounds record 80 actual Gemma requests for 80 responses; their cached schedule replays add no requests. Detailed local training and supporting extraction timings are listed in Appendix H-B.

TABLE VIII

Exact typed-rule coverage on RuleTest-94 and three archived CSV graph snapshots. All 18,143 compiled patterns are available. A zero in the raw graphs reflects missing node types, not verified absence of defects.

Input	Records	Typed	Matching rules	Flagged
RuleTest-94	94	94	2	10
Government	17,939	0	0	0
Finance	8,180	0	0	0
Environment	151	0	0	0

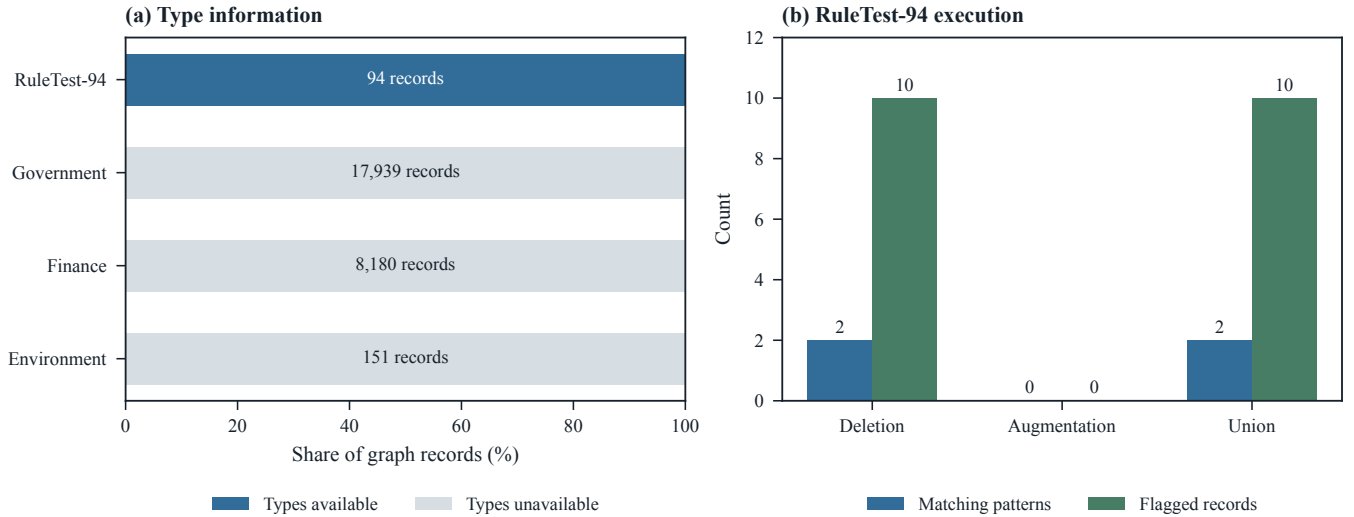


Fig. 8. Archive-rule integration audit. (a) Availability of endpoint types in each graph snapshot; numbers inside bars are record counts. (b) Exact matching patterns and flagged records on RuleTest-94. Missing type information causes abstention and is not scored as correct validation.

F. What the Three Stages Establish

The controlled scheduling study shows that reward design changes which repairs a learned policy completes. The rate reward improves over the original count penalty, and masking has a measurable component effect. The strongest simple heuristic still has the highest mean reference F1. The calibrated count control also shows why the observed reward gain cannot be attributed to normalization alone.

The rule studies explain what must hold when moving beyond that registry. Both strategies add candidates, but useful acquisition depends on applicable, correct rules that change repair options. Exact execution is limited by vocabulary coverage; the raw snapshots additionally lack types. With given types, generated type rules enable development repairs, yet the second-round candidates offer no final-F1 advantage from a different schedule. The remaining integration question is whether reliable generated rules create acquisition decisions for which learning improves quality or cost over simple controls.

Reference preservation in the scheduling study concerns controlled operators with trusted restoration records. The document development tests measure reference recovery after injected changes. Independent semantic review and new document domains are needed to assess natural-error repair. Supporting checks in Appendix G likewise distinguish structural conformance from extraction accuracy.

V. Conclusion and Future Work

We studied how to allocate a finite budget between acquiring validators and repairing a knowledge graph. The framework combines a Double-DQN scheduler with deletion-completion and augmentation-expansion rule generation. Our evaluation follows the connection from controlled scheduling, through candidate generation, to executable repair coverage.

In the fixed-registry study, counting temporary duplicates discourages useful relation restoration. Replacing that count penalty with an initial-size rate penalty raises Double-DQN reference-triple F1 from 94.67% to 99.49% on reserved corruption scenarios. The acquire-then-deficit heuristic reaches 99.91%. Current-rate ablations support the action mask: removing it lowers F1 by 2.50 points after multiplicity correction. A calibrated small count penalty is statistically indistinguishable from the rate penalty at the base size. These results identify reward and feasibility choices that affect repair, while the strongest simple schedule remains competitive with learning.

The generation study finds different candidates from deletion and augmentation on the same documents, with more constraints from augmentation at equal call budgets. Direct execution reveals a narrower result: both strategies together remove ten designed defects, but those removals are already covered by deletion alone. The document-based extension reaches 95.69% F1 in its second development round, removing seven injected items without reference losses. All seven removals use deletion-generated type rules, and the best feasible

schedule matches acquire-then-repair. The current candidates therefore offer no further final-F1 gain through scheduling in that development setting.

The next step is to establish reliable, complementary generated rules that create a meaningful acquisition decision, then compare learned scheduling with the same strong simple controls. Independent review of rules and edits, together with held-out document domains, would assess factual correctness and transfer beyond the controlled tasks.

Appendix A Generation Prompts and Recorded Settings

The supplementary artifact provides literal Chinese templates exported from the prompt builders, their SHA-256 hashes, and English translations. The translations below describe the same tasks and response fields; they are not presented as the language used in the historical calls. The archive contains the parsed responses and deletion fragments, but not each fully rendered request or a complete historical decoding configuration.

A. Deletion Completion Prompt

The actual builder asks: “You are an expert in rule extraction for government-service knowledge graphs. Some phrases have been randomly removed from a service document. Read the original and the incomplete version, infer the missing rule information using the context and domain knowledge, and express it as constraints for graph validation. Return only JSON following the supplied schema.” The request includes the service item, original source, incomplete source, and deleted fragments. It asks for general types and relations rather than only instance names.

The JSON fields are `strategy`, `service_item`, `analysis`, `removed_fragments`, `missing_rule_candidates`, and `proposed_rules`. The latter contains the seven candidate categories described below. This call is made once per document; ten repeated entity masks and frequency-threshold filtering are not part of the archived runner.

B. Augmentation Expansion Prompt

The actual builder asks: “You are an expert in rule extraction for government-service knowledge graphs. Propose no more than three reasonable supplementary clauses for the document, while preserving plausibility and compliance, and infer graph-validation rules from the additions. Return only JSON following the supplied schema.” The request includes the service item and original text, and asks for reusable rather than instance-specific candidates.

Its response contains `strategy`, `service_item`, `analysis`, `added_clauses`, and `proposed_rules`. The schema categories are entity types, relationship types, forbidden type-relation patterns, allowed type-relation patterns, hierarchy statements, geographical hierarchy patterns, and procedural statements. Both calls use the same candidate categories.

C. Defaults and Missing Metadata

The script defaults are Qwen3-32B, temperature 0.2, up to five deleted spans of at most six characters, and at most three supplementary clauses. The client does not explicitly set a completion-token cap or top- p . Historical per-call records contain the document identifier, service item, strategy, parsed result, and deletion fragments when applicable. They do not certify that every invocation used the current defaults. The separate 45-document extraction experiment records Qwen3.8-27B-no-thinking.

Appendix B RuleTest-94 and Direct Execution

The suite contains 30 clean and 64 defective records: 10 hierarchy reversals, 10 absurd relations, 10 reverse-supervision cases, 30 procedural or missing-field cases, and four specialist cases. The revised scorer reads `expected_detection` directly: `pass` is negative, while `fail` and `specialist_miss` are positive. It does not derive labels by running the detector under evaluation. These are designed labels without independent annotator identities or disagreement records.

The hand-implemented family union still detects all 64 defects without flagging the 30 clean records under these stored labels. Its precision Wilson interval is $[0.943, 1.000]$ and specificity interval $[0.886, 1.000]$. This family-union score is distinct from the direct execution of archived candidates reported in Section IV-D1.

Each directly compiled constraint has an identifier, allowed/forbidden kind, exact subject-type/relation/object-type pattern, and a list of original source locations. Pattern membership determines whether a record matches; allowed patterns do not define a closed-world whitelist. Contradictory permission and prohibition yield an explicit abstention. Unsupported natural-language candidates are retained without invented formal interpretations.

Appendix C Typed-Document Generation Pilot

We prepare a separate source-grouped DocRED [17] sample with 180 training, 60 development and 60 test documents from the original annotated training corpus. Entity identities and mention types are given. Public source inputs and relation references are stored separately; generation and policy observations do not read reference relations.

A fixed ten-document development pilot collects two rule responses and one natural extraction per document using Gemma. Its 30 outputs require 34 actual requests including retries. All twenty rule responses parse, yielding 263 compiled candidates: 156 allowed and 107 forbidden patterns. One extraction response fails the frozen parser and remains an empty graph; the others yield 106 executable relations. Five two-document episodes are replayed under immediate stop and four fixed acquisition/repair schedules. No schedule deletes a record, and all schedules produce the same final graph within each episode. The pilot therefore identifies a coverage gap between the generated forbidden patterns and these natural extraction outputs. Type compatibility alone does

TABLE IX

Source-evidence development diagnostics on ten fixed document pairs. F1 and preservation are episode means. Lost counts removed reference facts. No learned policy is evaluated.

Schedule	F1 (%)	Preserved (%)	Lost
Stop	94.42	100.00	0
Acquire then repair	94.59	98.34	4
Repair when feasible	91.63	93.51	18
Uniform feasible	94.42	100.00	0

not establish factual correctness. Formal learned scheduling on this new corpus has not started; the development records and implementation are archived in `exps/paper2__docred`.

A. Source-Evidence Development Test

A separate version retains dual-strategy generation and the four-action scheduler, and adds document-scoped source constraints. Each candidate marks a concrete record as supported, contradicted, or insufficient. The compiler validates an exact quote in a numbered original sentence and records its character offsets. Hypothetical augmentation clauses cannot supply evidence. Insufficient evidence causes abstention. Conflicting support and contradiction also cause abstention; type compatibility alone cannot cancel a source contradiction. For this version, let $V(g; R)$ be the set of nonconflicting violations and N the initial record count (with denominator one for an empty input). We use $G(g; R) = 1 - |V(g; R)|/N$ and coverage C over the initial records. The graph-change term compares both graphs under the updated rules:

$$\begin{aligned}
 r_t = & 0.5 [G(g_{t+1}; R_{t+1}) - G(g_t; R_{t+1})] \\
 & + 0.5(C_{t+1} - C_t) - 0.004a_t \\
 & - 0.00002d_t - |V_{\text{edit},t}|/N,
 \end{aligned} \tag{15}$$

where a_t counts acquired responses, d_t counts removed records, and $V_{\text{edit},t} = V(g_{t+1}; R_{t+1}) \setminus V(g_t; R_{t+1})$. Thus acquisition can reveal a violation without being charged for creating it. A two-record synthetic counterexample changes the discounted acquire-repair return from -0.266519 to 0.483481 at discount 0.95 . This is an accounting check, not an empirical quality gain. Source support is still a model judgment; quote validation does not prove its meaning. Before calls, we fix the first ten development pairs (20 documents). Each pair contains one reference graph and one reference graph with added endpoint or relation substitutions, numbering $\lceil 0.2|G^*| \rceil$. References are used in construction and scoring, not in prompts, observations, or reward. The observations contain 14 public summaries and an action mask; they are not asserted to be a sufficient Markov state. Each episode allows four response acquisitions, two per strategy, and ten steps. The removal operator retains the emptying guard. The 40 outputs require 40 actual Gemma requests. They compile to 357 type rules, 61 source-support rules, and 6 source-contradiction rules. Source constraints trigger removals in 4 of ten episodes. The pre-call expansion gate requires this in at least two episodes and at least one fixed schedule to improve mean reference F1 while preserving at least 99% of reference facts. The development

gate is not met, so this version is not expanded to learned-policy training or held-out evaluation. These results measure reference recovery and removal of injected items. Independent assessment of natural relations and edits remains pending. Cached schedule replay makes zero API calls; generation requests are counted separately.

a) *Second development round.*: We retain the same twenty documents, compiler, reward, budget, and gate. The only changes add readable entity names and clarify property direction, historical roles, and the distinction between absent and contradictory evidence. Forty further requests complete without transport failures; 38 responses pass the fixed rule parser. They yield 243 type rules and 14 source-support rules, but no executable source-contradiction rule. Both acquire-then-repair and repair-when-feasible reach 95.69% reference F1, compared with 94.42% for stop. Each removes seven of 31 injected items and loses no reference fact. All seven removals come from type constraints. Source constraints therefore fail the required repair-effect gate, and the two development rounds end without expanded training or a held-out generated-rule policy evaluation. The compiler also rejects 120 and 257 type candidates in the two rounds that copy the example string allowed or forbidden instead of selecting one kind. The fixed parser is not relaxed after collection. The first round’s harmful deletions and the second round’s empty source-repair coverage are both retained. These checks show why executable quotation and correct scheduling interfaces alone do not establish useful source constraints.

Appendix D

Frozen Reward-Validation Protocol

Training uses seeds 20000–20009, with environment seed $100000s + e$ for seed s and episode $e \in \{0, \dots, 249\}$. Separate feasible-random ridge rollouts use seeds 30000–30009 with the same episode formula. Development scenarios are 920000–920019; reserved test scenarios are 930000–930029. These sets are disjoint from the earlier ten audit scenarios. All scenarios share the same 450-document base graph.

The training manifest fixes sources, graph data, the three penalty definitions, four neural settings, ridge specification and statistical comparisons. It selects no checkpoint or penalty using development scores. After all forty neural checkpoints, ten ridge fits and development checks are complete, a second manifest seals their hashes and the scoring code before test execution. The test evaluates all eight predefined policies. Each run uses the same thirty scenarios, allowing paired comparisons while keeping the ten run-seed means as statistical units.

Scoring is separate from action selection. Policies receive the public observation and current mask; neither clean-reference triples nor the private relation-restoration record enters their inputs. Saved initial graphs and stepwise node/edge deltas support reconstruction of every trajectory. The artifact checks final reference metrics, residual defects, masks, reward decomposition, cumulative costs and all three counterfactual returns against those reconstructed trajectories. No environment probes occur during inference.

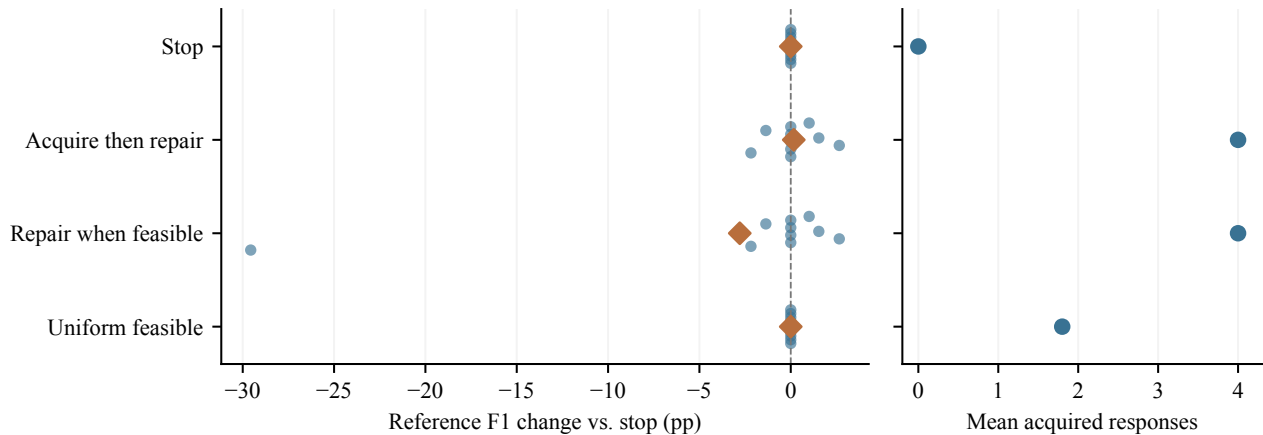


Fig. 9. Source-evidence development replay. Small points show ten episode-level F1 changes from stop; diamonds show their mean. The right panel shows cached response acquisitions.

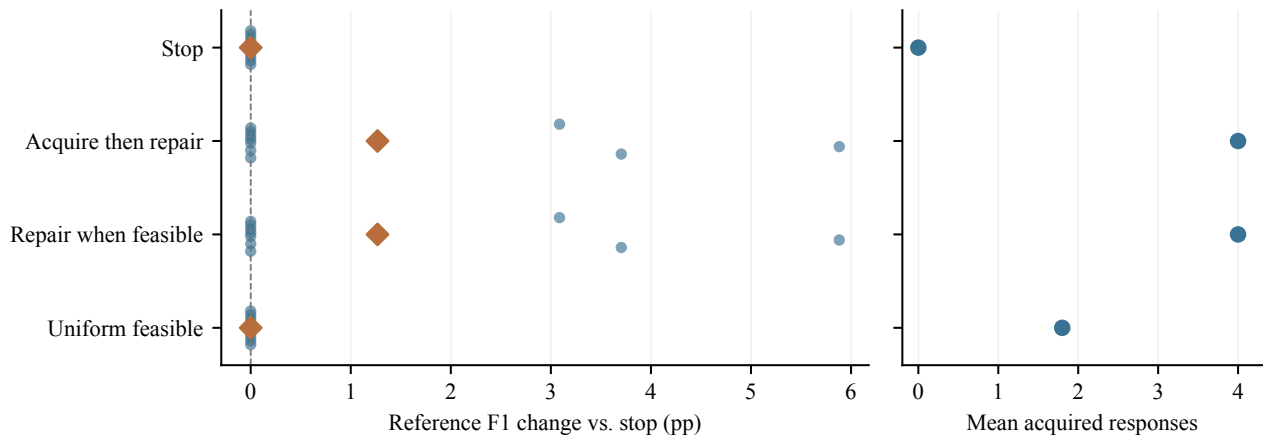


Fig. 10. Second and final development round on the same ten pairs. F1 improves under the two repair schedules without reference losses, but all removals use type constraints. Repeated rounds are not new test documents.

A. Training and Reproducibility Records

The reward study freezes training sources, costs, seeds and episode budgets before training forty neural models and ten ridge baselines. Development checks cover 1,600 policy/run/scenario outcomes. After those checks, a second manifest seals all fifty fitted models and the scoring code before the thirty reserved test scenarios are opened, yielding 2,400 test outcomes. All predefined policies are retained, with no coefficient search or development-based selection. Initial graphs, stepwise graph deltas, masks, reward components and reference scores are archived. Earlier environments, checkpoints and results remain available under their original versions. Reward-study training and cached evaluation make zero API requests. The earlier extraction test used Qwen3.8-27B-no-thinking; historical rule-generation settings are described in Appendix A.

Appendix E

Earlier Count-Penalty Component Study

The observation order is: four graph components; rule precision, recall and coverage; four normalized defect counts;

remaining budget; and the active fractions of high- and low-precision registered modules. The action order is isolation cleanup, deduplication, relation repair, dangling-edge cleanup, deletion acquisition, augmentation acquisition, mixed acquisition, and pruning.

This earlier comparison contains sixty trained models and 110 evaluations. It uses the ten original scenarios 900000–900009. All sixty revised models (DQN, Double DQN, and four ablations) use training seeds 10000–10009, the original 250-episode budget, network dimensions, replay settings, and 260-episode exploration schedule. The graph-feature ablation zeros observation indices 0–3 and 7–10; the rule-feature ablation zeros indices 4–6 and 12–13, using zero-based indexing. Other inputs and the feasibility mask remain available. These are explicit-feature ablations, since the mask can itself convey rule availability.

The no-mask variant removes the behavioral and Bellman masks. The no-call-penalty variant adds back the accounted acquisition penalty during training. Every evaluation in this component study uses the count penalty, unchanged cost coefficients, and 18-step budget. Early terminal quality is

carried forward to step 18 when computing trapezoidal AUC. The revision keeps the original short-trajectory AUC in its historical archive rather than mixing the two definitions.

Two-sided paired sign randomization enumerates all 2^{10} sign assignments. Confidence intervals use 10,000 whole-scenario bootstrap draws with seed 42. Holm correction covers ten comparisons: full Double DQN versus the fixed strong heuristic and each of four ablations, for final quality and common-budget AUC. The same scenarios were evaluated previously, so these follow-up comparisons do not constitute a new held-out test.

A. Earlier Policy Results

With the count penalty, Double DQN reaches mean final quality 0.96855 and AUC 0.94514, using 6.2 accounted calls. The acquire-then-deficit heuristic reaches 0.98248 and 0.95797 with four calls. Full Double DQN minus the heuristic gives differences of -0.01392 and -0.01283 ; both Holm-adjusted p values are 0.0195 (Table XII). Standard DQN reaches final quality 0.96920.

The reward now charges for introduced violations. Double DQN introduces none on these ten evaluation runs, while the heuristic introduces 4.8 per run on average. For $\mathcal{R} = \sum_{t=0}^{T-1} 0.95^t r_t$, the mean discounted returns are 0.3660 and 0.3508, respectively. The action audit in Section IV-B shows that the zero count comes from avoiding relation repair. These returns are under the count penalty, separately from the common rate-penalty evaluation in the main text. All reward components and introduced identities are retained for each transition.

B. Retrained Component Ablations

We train ten models for each of four variants, using the original 250 episodes per seed. The variants remove explicit graph features, explicit rule features, the action mask, or the acquisition-call penalty. Feature removal applies during both training and evaluation. The mask remains in the feature ablations and can convey rule availability. The no-mask variant removes masking from action selection and Bellman targets. The no-call-penalty variant changes only the training reward; evaluation uses the same corrected reward and cost coefficients for all settings.

Removing the mask lowers final quality to 0.94800 and produces 10.7 invalid actions per run. Its final-quality contrast passes Holm correction ($p = 0.0195$); its AUC contrast does not ($p = 0.8340$). The graph-feature, rule-feature, and call-penalty contrasts do not pass the ten-test correction. Thus the revised results support the action mask’s contribution to final quality, while the other independent gains remain unestablished in this environment.

The analysis enumerates all 2^{10} sign assignments for exact two-sided paired randomization tests and applies Holm correction across ten contrasts: full Double DQN against the heuristic and four ablations, for both metrics. Pointwise intervals use 10,000 paired bootstrap draws. The sixty new models contribute 15,000 completed training episodes. Earlier checkpoints and results remain archived under their original environment version.

Appendix F Artifact Index

Original checkpoints, transitions, and generation records remain under their original experiment directories. The earlier offline study is in `exps/paper2_offline_revision/`. It contains frozen source hashes, forty new checkpoints and training histories, all paired evaluation transitions, equal-call candidate samples, rule provenance, explicit label checks, and publication generators. The manuscript-to-artifact audit and offline reproduction commands accompany the results. These local analyses and training runs make zero API calls; the document-based generation requests are recorded separately.

The corrected environment, sixty retrained checkpoints, complete evaluation trajectories, shared-profile Paper1 revision, and rule bridge are in `exps/math_revision_20260925/`. Its source manifest and offline reproduction entry point accompany the results.

The reward-validation study is in `exps/paper2_reward_validation/`. It contains training and test manifests, forty neural checkpoints, ten ridge models and their public-feedback rollouts, development and test trajectories, paired analyses, reconstruction checks, and generators for the active reward figures and tables. Its README gives commands that require no API requests or model downloads.

The separate policy-interface and coverage audit is in `exps/paper2_rule_integration/`. It pins the raw CSV snapshots, retains matching and nonmatching rules, and records sixteen schedule traces. Public runtime records and scorer-only suite labels are stored separately. Synthetic interface tests are excluded from empirical denominators.

a) *Offline scheduling diagnosis.*: The follow-up in `exps/papers_readiness_20261002/` enumerates every feasible acquisition, repair and stop sequence under the existing four-packet budget and ten-step horizon. It preserves the two development rounds and their twenty shared documents. References score paths only after enumeration. Table XIII separates an executable schedule oracle from a relaxed upper bound that may individually remove any injected record reachable under the stored proposals.

In round 1, the executable oracle reaches 95.45% F1, but paths maximizing the proxy reward reach 91.63%, below stopping at 94.42%. Wrong rules can therefore misalign the proxy objective even with corrected reward accounting. In round 2, both oracle bounds reach 95.69%, equal to acquire-then-repair. All seven removable injected items are covered by deletion-generated type rules; augmentation and source constraints add no removal opportunities. Once all packets are acquired and feasible repairs completed, round-2 final graphs are identical across orders. Discounted returns still vary with order; this leaves a cost and timing question, rather than additional final-F1 headroom on these fixed candidates. The 598 terminal paths are post-hoc development diagnostics, not new documents or learned-policy evaluations.

An offline output-contract draft supplies separate legal enum examples, explicit families, candidate limits and empty-result behavior. Replaying all 80 historical responses with the

TABLE X
Corrected-environment policy comparison over ten paired scenarios (mean $\pm$ sample standard deviation). Calls are accounted acquisition units.

Policy	Final quality	AUC ₁₈	Calls	Invalid	New viol.	Return
Random valid	0.9642 $\pm$ 0.0065	0.9254 $\pm$ 0.0152	10.2	0.0	2.9	0.3163
Alternating valid	0.9677 $\pm$ 0.0007	0.9292 $\pm$ 0.0020	10.0	0.0	4.0	0.3102
Rule-first valid	0.9747 $\pm$ 0.0004	0.9365 $\pm$ 0.0024	8.0	0.0	4.6	0.3255
Acquire + deficit	0.9825 $\pm$ 0.0001	0.9580 $\pm$ 0.0014	4.0	0.0	4.8	0.3508
DQN	0.9692 $\pm$ 0.0037	0.9454 $\pm$ 0.0079	5.2	0.0	0.0	0.3688
Double DQN	0.9686 $\pm$ 0.0034	0.9451 $\pm$ 0.0096	6.2	0.0	0.0	0.3660
Model-informed lookahead	0.9711 $\pm$ 0.0048	0.9516 $\pm$ 0.0027	5.1	0.0	0.2	0.3750

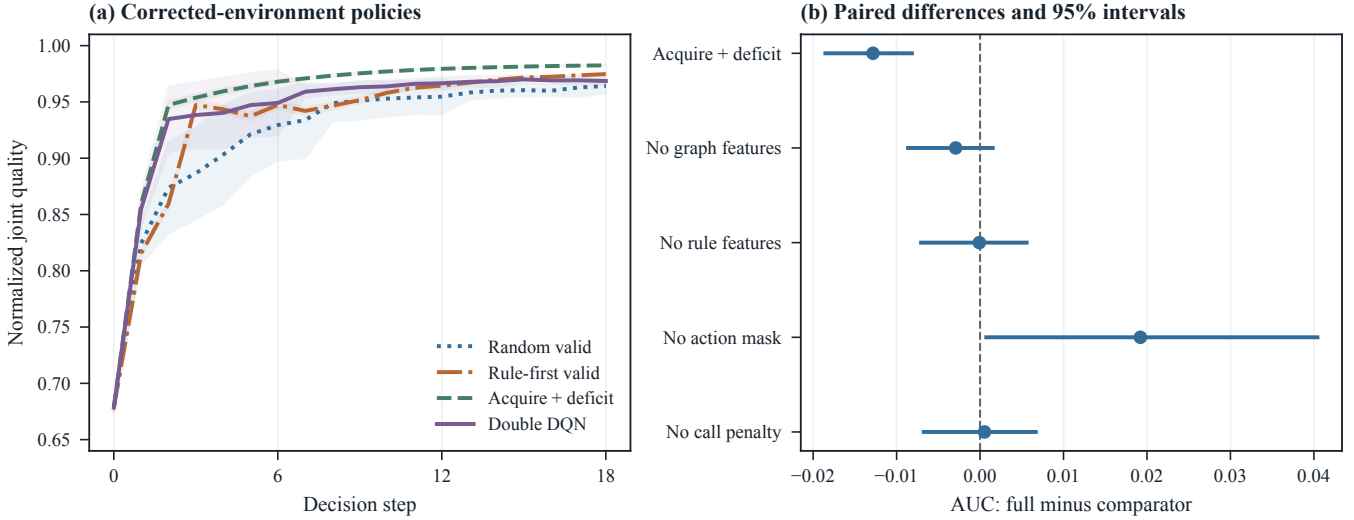


Fig. 11. Earlier count-penalty policy study on ten reused scenarios. (a) Mean evaluation quality with one-standard-deviation bands over ten scenarios. Terminal scores are carried forward to step 18. (b) Full Double DQN minus each comparator in AUC, with pointwise 95% paired-bootstrap intervals. Multiplicity-adjusted tests appear in Table XII.

TABLE XI
Corrected-environment ablations, ten retrained models per setting.

Setting	Final quality	AUC ₁₈	Calls	Invalid	New viol.	Return
Double DQN	0.9686 $\pm$ 0.0034	0.9451 $\pm$ 0.0096	6.2	0.0	0.0	0.3660
No graph features	0.9713 $\pm$ 0.0041	0.9481 $\pm$ 0.0028	5.1	0.0	0.9	0.3693
No rule features	0.9692 $\pm$ 0.0062	0.9452 $\pm$ 0.0034	7.6	0.0	0.5	0.3594
No action mask	0.9480 $\pm$ 0.0215	0.9259 $\pm$ 0.0278	4.9	10.7	0.0	0.3430
No call penalty	0.9659 $\pm$ 0.0051	0.9446 $\pm$ 0.0044	8.1	0.0	0.0	0.3596

unchanged compiler reproduces their original rules and rejections. The draft has not been used for generation; the failed expansion gate and unrun formal study remain unchanged.

Appendix G Supporting Repair-Layer Checks

These checks evaluate detector coverage, structural repair, and extraction behavior. The learned-policy comparisons are reported in Section IV-B; generated-rule effects are reported in Section IV-D.

A. Separate Rule-Family Check

The hand-implemented deletion-family detector covers 34 procedural, missing-field, and specialist defects. The augmentation-family detector covers the other 30 structural defects. Their union detects all 64 defects with no false

positives when scored against the stored expected labels. These functions encode the designed defect families directly. Their coverage establishes a regression check for those families; the direct execution results in Table VI measure the archived generated patterns.

B. Comparison with Constraint-Checking Baseline

We compare with a deterministic constraint-checking baseline implemented with SHACL-style RDF validation. This baseline receives deduplicated RDF triples and checks schema-conformance constraints, but it does not access the source text, LLM prompts, or generated procedural/value rules. This comparison measures the coverage of the selected shapes on these inputs.

The inputs conform to the selected shapes after RDF deduplication, so the baseline removes no additional triples. Its

TABLE XII

Full Double DQN minus each comparator in the corrected environment. Intervals are pointwise paired-bootstrap 95% intervals; Holm adjusts ten tests.

Comparator	Metric	Difference	95% interval	p_{Holm}
Acquire + deficit	Final	-0.01392	[-0.01604, -0.01205]	0.0195
Acquire + deficit	AUC	-0.01283	[-0.01853, -0.00817]	0.0195
No graph features	Final	-0.00273	[-0.00601, -0.00007]	0.8340
No graph features	AUC	-0.00292	[-0.00862, +0.00151]	1.0000
No rule features	Final	-0.00068	[-0.00406, +0.00268]	1.0000
No rule features	AUC	-0.00008	[-0.00706, +0.00558]	1.0000
No action mask	Final	+0.02055	[+0.00897, +0.03502]	0.0195
No action mask	AUC	+0.01921	[+0.00075, +0.04042]	0.8340
No call penalty	Final	+0.00269	[-0.00031, +0.00572]	0.8340
No call penalty	AUC	+0.00053	[-0.00674, +0.00667]	1.0000

TABLE XIII

Post-hoc exhaustive scheduling on the same ten development pairs per round. F1 is averaged over pairs. Oracles use reference labels only for analysis.

Diagnostic	Round 1	Round 2
Feasible terminal paths	363	235
Stop F1 (%)	94.42	94.42
Executable oracle F1 (%)	95.45	95.69
Selective-removal upper F1 (%)	95.60	95.69
Reward-optimal F1 (%)	91.63	95.69
Oracle correct facts lost	0	0
Oracle injected items removed	5	7

TABLE XIV

Hand-implemented family detectors, rescored against stored suite labels. These detectors are separate from the compiled candidate patterns.

Detector	TP	FP	FN	Recall	F1
Deletion-family	34	0	30	0.531	0.694
Augmentation-family	30	0	34	0.469	0.638
Family union	64	0	0	1.000	1.000

average quality score is 71.85 under the separate quality metric. These shapes do not implement all source-dependent and procedural checks used by that metric. This is a difference in implemented constraint coverage; SHACL can express richer value and logical constraints than this baseline uses.

C. Deterministic Structural Repair

A separate deterministic benchmark uses TNEWS/CLUE-style Chinese news titles. A deterministic extractor constructs a document–category–entity KG from 3,000 balanced news titles across 15 categories; we then inject duplicate triples, invalid relations, and isolated nodes, and run the local repair pipeline. This benchmark is fully local and makes no LLM/API calls, so it evaluates KG quality control and rule execution rather than LLM extraction quality.

The repair pipeline removes injected invalid relations, duplicate triples, and isolated noise nodes, improving structural Q from 94.93 to 100.00. The repaired graph has fewer nodes and relations than the clean extraction, so this score does not mean that every original fact was recovered. The full report is generated by `exps/external_benchmark_runner.py`.

TABLE XV

SHACL-style structural validation baseline on three domains.

Domain	Raw Triples	RDF Dedup.	Removed	Q
Gov. Affairs	13,892	10,771	0	79.87
Finance	2,264	539	0	70.42
Env.	2,345	378	0	65.25
Average	6,167	3,896	0	71.85

TABLE XVI

TNEWS/CLUE local end-to-end KG quality benchmark.

Stage	Nodes	Relations	Invalid	Q
Clean extraction	10,871	20,827	0.000	99.89
Corrupted KG	11,414	22,909	0.050	94.93
Repaired KG	10,822	19,799	0.000	100.00

D. API-Backed LLM Extraction Benchmark

The local TNEWS/CLUE benchmark above isolates the quality-control layer under deterministic extraction. To additionally test the full extraction path, the archived API benchmark evaluated on 45 TNEWS/CLUE titles covering 15 categories. The model is Qwen3.8-27B-no-thinking. TNEWS category labels are used as silver labels for category inference, and provided title keywords are used as weak silver mentions for entity coverage. The test measures parsing, weak extraction indicators, and structural filtering of model outputs.

The 45 calls completed in 1,190.90 seconds and all responses parsed successfully. Qwen3.8-27B-no-thinking followed the allowed schema on every sampled title, so the conservative repair layer made no change: raw and repaired structural quality are both 100.00. No structural violation was detected on this sample. Category accuracy is 0.556 and weak keyword recall is 0.178, indicating that semantic extraction remains the larger bottleneck; deterministic structural checks cannot repair omitted or misclassified content.

Appendix H

Bridge Coverage and Runtime Records

A. Types and Fixed-Schedule Traces

The bridge in Section III-E provides a public-observation interface for loading either strategy’s archived rule bank, removing unopposed forbidden matches, or stopping. Two

TABLE XVII
API-backed LLM extraction benchmark on TNEWS/CLUE titles.

Metric	Raw LLM KG	Repaired KG
Parse success	1.000	—
Category accuracy	0.556	0.556
Keyword recall	0.178	0.178
Documents with triples	1.000	1.000
Avg. triples / doc	3.58	3.58
Invalid triple rate	0.000	0.000
Duplicate triple rate	0.000	0.000
KG quality score	100.00	100.00

acquisition orders, each with immediate or deferred removal, give four schedules per input. Across RuleTest-94 and three archived graph snapshots, we retain sixteen complete action traces. All four schedules remove the same ten designed defects and no clean record on RuleTest-94. No late permission or conflict occurs in these empirical replays. Separate synthetic tests verify conflict abstention and log a permission that arrives after a record was removed; those tests do not add empirical cases.

The original government, finance and environment CSV snapshots contain 17,939, 8,180 and 151 edge occurrences, respectively. Their node types are all Unknown, leaving 26,270 edges without the type information required by the archived patterns. A further provenance audit checks 51 tracked node CSV files, the three original document JSONL files and two separate-task extraction checkpoints. It restores no traceable types for these graphs. In the enhanced exports, Enhanced is assigned by the converter as a processing tag, not an entity class; numeric IDs alone do not establish identity across exports. The runtime abstains on these records. Table VIII distinguishes this missing information from the absence of a matching rule on typed records. These are separately pinned raw snapshots, not the inputs of the SHACL comparison in Appendix G-B.

On RuleTest-94, the two matching patterns both come from deletion: one permission matches ten records and one prohibition flags another ten. Augmentation matches no record. The resulting coverage and identical final records across schedules identify the next integration requirement: trusted entity types and broader applicable rule coverage before testing learned scheduling. The replay interface provides no reward and uses no learned checkpoints. Loading a strategy bank is reported as loading 4,728 archived responses, with zero actual API requests.

B. Runtime Details

The original twenty-model RL run recorded 1,278.99 seconds of CPU time with zero API calls. The earlier sixty-model component study and the new forty-model reward study also make zero requests. The new ridge controls add ten fits from 2,500 random exploration episodes. The current-reward component and small-count study adds fifty trained models, again with zero API requests. Per-run training durations and evaluation timings are archived. Evaluation timing covers policy inference and environment transitions, excluding environment construction. Model-informed lookahead additionally

includes cloned transition probes. These local timings do not estimate LLM latency.

The earlier 45-document extraction test made 45 actual calls and recorded 1,190.90 seconds. The rule archive records 9,456 successful responses; historical latency, tokens, retries, and failed requests are not fully recorded. Equal-call candidate comparisons therefore measure diversity per archived successful call, rather than total billed cost. The environment’s acquisition units model that cost during scheduling.

References

- [1] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, and A. McCallum, “Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning,” in *International Conference on Learning Representations*, 2018.
- [2] W. Xiong, T. Hoang, and W. Y. Wang, “DeepPath: A reinforcement learning method for knowledge graph reasoning,” in *Proceedings of EMNLP*, 2017.
- [3] X. V. Lin, R. Socher, and C. Xiong, “Multi-hop knowledge graph reasoning with reward shaping,” in *Proceedings of EMNLP*, 2018.
- [4] L. Yao *et al.*, “Kg-bert: Bert for knowledge graph completion,” in *arXiv preprint arXiv:1909.03193*, 2019.
- [5] X. Wang *et al.*, “Kepler: A unified model for knowledge embedding and pre-trained language representation,” in *Transactions of the Association for Computational Linguistics*, 2021.
- [6] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [7] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, 2022.
- [8] S. Yao *et al.*, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [9] H. Knublauch and D. Kontokostas, “Shapes constraint language (shacl),” *W3C Recommendation*, 2017.
- [10] E. Prud’hommeaux *et al.*, “Shape expressions: An rdf validation and transformation language,” *Proceedings of the 10th International Conference on Semantic Systems*, 2014.
- [11] L. A. Galárraga *et al.*, “Amie: Association rule mining under incomplete evidence in ontological knowledge bases,” in *Proceedings of the 22nd International Conference on World Wide Web*, 2013.
- [12] S. Ortona *et al.*, “Rudik: Rule discovery in knowledge bases,” in *Proceedings of the VLDB Endowment*, 2018.
- [13] F. Yang *et al.*, “Differentiable learning of logical rules for knowledge base reasoning,” in *Advances in Neural Information Processing Systems*, 2017.
- [14] T.-W. Lin, G. Fierro, H. Li, T. Hong, P. Nuzzo, and A. Sangiovanni-Vincentelli, “Systematic evaluation of knowledge graph repair with large language models,” *arXiv preprint*, 2025.
- [15] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [16] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.
- [17] Y. Yao, D. Ye, P. Li, X. Han, Y. Lin, Z. Liu, L. Huang, J. Zhou, and M. Sun, “DocRED: A large-scale document-level relation extraction dataset,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 764–777.